Page No.
Date:

Good Afternoon,

My name is Lakshay Gupta and I am currently undergoing my training as an Analyst Trainee at Cognizant, Chennai.

During my training, I have gained foundational knowledge in Java, SQL, Manual Testing, and Selenium.

In Java, I learned core concepts such as classes and objects, OOPs Concepts, Exception Handling, and Collection Framework including List, Set and Map. and have theoritical knowledge about

In Manual Testing, I learned, SDLC, STLC, Functional and Non-Functional Testing, and Defect Life Cycle.

In Selenium, I learned the basics of Webdriver, X path, locators, Waits and Automation Testing concepts.

And currently I am focused on strengthening my knowledge and Technical skills in automation Testing.
Thank You.

Java

Java is a high level, class based, & Object Oriented programming language.

JDK ⇒ Java Development kit, it includes JVM and used for running Java Programs.

JRE ⇒ Java Runtime Environment, it provides the environment to run Java applications.

JVM ⇒ Java Virtual Machine, it is used to convert the bytecode into machine code.

Class ⇒ A class in Java is a blueprint from which objects are created.

Object ⇒ An object is an instance of a class, it contains data and methods.

ex ⇒ Student = class, s1 = object.

Method ⇒ A method is a block of code that performs a specific task.

ex ⇒ public static void main (String[] args) {
// block of code
}

Constructor ⇒ It is a special method that is automatically called when an object of an class is created.

ex ⇒ class Student {

Default Constructor

Student () {
// block of code
}

Parameterized Constructor

Student (String name, int age) {
}

Copy Constructor Student (Student s) {

}

↳ It initialize a new object using the values of an existing object.

OOP's Concepts

Object Oriented Programming System is a programming approach where we create programs using classes & objects.

There are four concepts :-

1. Encapsulation :- Wrapping variables and method into a class and protecting the data using access modifier.

ex:-
```
class Student {
    private String name;

    public void setName(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
```

- Inheritance ⇒ One class acquires the properties and methods of another class. using extends keywords.

ex ⇒
```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {

}
```

- Polymorphism ⇒ It means one method having multiple forms. It is achieved using method overloading and method overriding.

ex ⇒
```
class Dog extends Animal {

    void eat() {
        System.out.println("Dog Eating");
    }
}
```

- Abstraction ⇒ Showing only important information and hiding implementation details.

ex ⇒ abstract class Vehicle {
    abstract void start();
}

class Car extends Vehicle {
    void start() {
        System.out.println("Car Started");
    }

Example

```
abstract class Vehicle {                    // Abstraction
    abstract void start();
}

                                            // Inheritance
class Car extends Vehicle {
    private String brand;                   // Encapsulation

                                            // Constructor
    Car(String brand) {
        this.brand = brand;
    }
                                            // Getter Method
    public String getBrand() {
        return brand;
    }

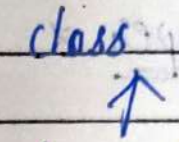
    @Override                               // Method Overriding
    void start() {
        System.out.println("Car g Started");
    }

public class Main {
    public static void main(String[] args) {

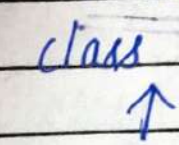
        Car car = new Car("Honda");
        System.out.println(car.getBrand());
        car.start();
    }
}
```

Page No.
Date:

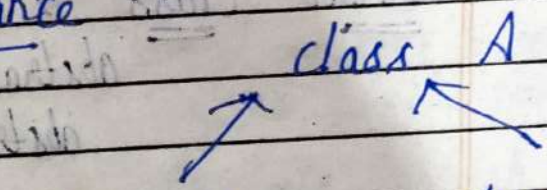
Single Inheritance



Multilevel Inheritance

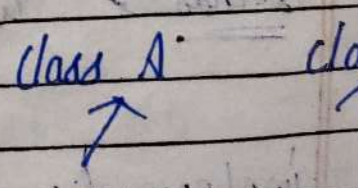


Hierarchical Inheritance



Multiple Inheritance

→ It is implemented using Interface in Java using implements keywords.



- **final** ⇒ It is used to make something unchangeable. It can be variable, method, class.
- **Super** ⇒ It refers to the immediate parent class objects.
- **Method Overloading** ⇒ Same methods name with different parameters.
- **Method Overriding** ⇒ child class provides its own implementation of a method that already exists in the parent class.
- **Abstract class** ⇒ A class which is declared with the abstract keyword is known as abstract class.
- **Interface** ⇒ An interface is a blueprint that contains only method declarations. It tells class what to do, but not how to do it.
- **this** ⇒ Refers to the current object.
- **extends** ⇒ used when one class inherits another class.
- **implements** ⇒ used when a class implements an interface.
- **Static** ⇒ it is used when a variable, method, or block belongs to the class rather than an object.

Access Modifiers

⇒ In Java, it defines the scope of classes, methods, and variables.

They determine who can access what in your code.

- Public — from any class in any package.
- Protected — within the same package, and by child classes in other package.
- Private — only accessible within the same class.
- Default — when no access modifier is specified (within the same package)

String ⇒ A String is immutable, which means once created, its value cannot be changed.

StringBuffer ⇒ StringBuffer is mutable, its value can be changed.

StringBuilder ⇒ It works like StringBuffer, but it is faster.

Array ⇒ It is a container object that holds fixed number of values of the same type.

Declare ⇒ `int[] numbers = new int[5];`

Initialize ⇒ `numbers[0] = 10;`
`numbers[1] = 11;`

Page No.
Date:

Data Types

⇒ A Data type tells Java what type of value a variable can store

Primitive Data Types	Non-Primitive Data Types
Int	String
double	Array
char	Class List
boolean	Interface Set
Float	Map
⇒ It store simple values directly	⇒ It stores references to objects

Exception Handling

⇒ Exception Handling is used for handling abnormal situations during program execution

ex ⇒
```
Int a = 10;
Int b = 0;

System.out.println(a/b);
```

⇒ It will show Arithmetic Exception.
⇒ Because division by zero is not allowed.

- try ⇒ The code that may cause an exception is written inside the try block.

ex ⇒
```
try {
    int result = 10 / 0;
}
```

- catch ⇒ It handles the execution.

ex ⇒ after the try block
```
catch (Exception e) {
    System.out.println("Exception handled");
}
```

- finally ⇒ It always executes whether an exception occurs or not.

ex ⇒ after the try-catch block
```
finally {
    System.out.println("Finally block");
}
```

- throw ⇒ throw is used to manully create an exception.

ex ⇒
```
if(age<18) {
    throw new ArithmeticException("Not Eligible");
}
```

Page No.
Date:

- throws ⇒ It is used in the method declaration to tell Java that a method may throw an exception.

ex ⇒ void readFile() throws IOException {
System.out.println("Reading File");
}

- Checked Exception ⇒ They are checked by the compiler at compile time.

ex ⇒ IOException, SQLException, FileNotFoundException

- Unchecked Exception ⇒ They occurs during runtime. and are not checked by the compiler.

ex ⇒ Arithmetic, NullPointer, ArrayIndexOutOfBound.

Collections

In Java, A collection is a framework (set of classes and interfaces) that's storing and manipulating group of objects.

The main collections are List, Set and Map.

* A map stores key-value pairs with unique keys.

- Needed to handle dynamic size of data.
- Needed to perform operations.
- To use data-structures.

* List → It stores elements in the order they are added.
- Allows Duplicate, Maintain Insertion Order.

ex → List<String> names = new ArrayList<>();

names.add("A");
names.add("B");

Sout(names);

* Set → A set stores unique elements.

ex →
```
Set<String> names = new HashSet<>();
names.add("  ");
names.add("  ");
sout(names);
}
```

* Map → A map stores data in a key-value pairs.

key must be unique.
Value can be duplicate.

ex →
```
Map<Integer, String> students = new Hashmap<>();
students.put(101, "Lakshay");
students.put(102, "Lakshay");
sout(students);
}
```

ex →
```
for(Map.Entry<Integer, String> entry : students.
    entrySet()) {
sout(entry.getKey());
sout(entry.getValue());
}
```

SQL

- SQL (Structured Query language) is a language used to store, retrieve, update, and delete data from a database.

- Database ≠ Organised collection of data.

- Schema ≠ It is a logical structure that contains database objects such as: Table, Views, Indexes, Functions.

- Table ≠ A table stores related data in rows & columns.

- Column ≠ It represents a field in a table and stores one type of information.

- Row ≠ It represents a single record in a table.

SQL Commands

1. DDL (Data Definition language)
   ≠ Used to create and modify database objects like tables.
   ≠ CREATE ≠ Creates a new table.

- ALTER ⇒ Modifies an existing table.
- DROP ⇒ Delete the entire table.
- TRUNCATE ⇒ Delete the all rows from a table.

2. DML (Data Manipulation Language)

⇒ Used to manipulate data inside tables.

- INSERT ⇒ Adds data.
- UPDATE ⇒ Modifies existing data.
- DELETE ⇒ Deletes records.

3. DQL (Data Query Language)

⇒ Used to retrieve data

- SELECT ⇒ Retrieve all records.
- WHERE ⇒ Filters records

4. TCL (Transaction Control Langua

⇒ Used to manage transactions

- COMMIT ⇒ Saves changes permanently
- ROLLBACK ⇒ Undoes changes.

5. DCL (Data Control Language)

⇒ Used to manage permissions.

- GRANT ⇒ Gives access.
- REVOKE ⇒ Removes access.

Primary key ⇒ It is a column that uniquely identifies each row in a table.
- must be unique, cannot be null, only one primary key per table.

Foreign key ⇒ It is a column that creates relationship between two tables.

Subqueries ⇒ query inside another query.

ex ⇒ Select * from employee
Where salary = (
Select Max(salary)
From Employee
);

Example

- Create Tables (DDL)

```
CREATE TABLE Department (
  DEPTID INT PRIMARY KEY,
  DEPTName varchar(50)
  );

CREATE TABLE Employee (
  EmpID INT PRIMARY KEY,
  EmpName VARCHAR(50);
  Salary INT,
  DeptID INT,
  FOREIGN KEY ( DeptID)
  );
```

- DeptID = Primary key in Department table.
- EmpID = Primary key in Employee table.
- DeptID in Employee table = Foreign key.

- INSERT Data (DML)

```
INSERT INTO Department
Values ( 1, 'Testing');

INSERT INTO Department
Values ( 2, 'Automation');

INSERT INTO Employee
Values (101, 'Lakshay', 50000, 1);

INSERT INTO Employee
Values (102, 'Rahul', 6000, 2);
```

- SELECT Data (DQL)

```
SELECT * FROM Employee;
```

output
101 Lakshay 50000 1
102 Rahul 6000 2

- WHERE Clause

```
SELECT * FROM Employee
WHERE EmpID = 101;
```

output
101 Lakshay 50000 1

- UPDATE Data

```
UPDATE Employee
SET Salary = 55000.
Where EmpID = 101;
```

output
101 Lakshay 55000

DELETE data

⇒ DELETE from Employee
Where EmpID = 102;

ORDER By

⇒ Select * from Employee
ORDER By Salary ASC;
= Ascending
DSC;
= Descending

ALTER Table (DDL)

⇒ ALTER TABLE Employee
ADD City VARCHAR(50);

TRUNCATE (DDL)

⇒ TRUNCATE TABLE Employee;

• Employee Table exists but Rows = 0

DROP Table (DDL)

⇒ DROP TABLE Employee;
Employee Table Deleted Completely.

- GROUP By ⇒ It forms groups of matching values.

- Find Total Salary Department Wise

⇒ Select Dept, SUM(Salary)
From Employee
GROUP BY Dept;

- HAVING ⇒ It is used to filter grouped data.

⇒ Select Dept, Sum(Salary) From Employee
Group By Dept
Having SUM(Salary) > 120000;

- WHERE And HAVING

	↓	↓
⇒	Filters individual rows before grouping	Filters groups after grouping.
⇒	Select * From Employee WHERE Salary > 50000;	Select * From Employee GROUP BY Dept Having SUM(Salary) > 12000;

Joins

⇒ Joins are used to combine data from two or more tables based on a related column.

⇒ Suppose we have two tables ⇒
Employee Table { Department Table
⇒ Both Tables have { Dept-ID
We use Joins to combine data from both tables.



1. Inner Join
⇒ Returns only matching records from both tables.



2. Left Join
⇒ All records from left table, matching records from right table.



3. Right Join
⇒ All records from right table, matching records from left table.



4. Full Outer Join
⇒ All matching and non-matching records from both tables

Page No.
Date:

- Self Join
= A table joins with itself.

Query for Second Highest Salary

Select Max(salary) As second highest salary
From Employee
Where salary < (Select Max(Salary) From Employee);

Manual Testing

- Testing :- It is the process of checking whether the application is working as expected or not.

- Manual Testing :- It is the process of testing an application without using automation tools.

SDLC (Software Development Life Cycle)

:- It is the process followed to develop a software (SDLC describes how software is developed)

- Requirement Gathering
- Analysis
- Design
- Development
- Testing
- Deployment
- Maintenance

★ STLC (Software Testing Life Cycle)

→ STLC is the testing process (it describes the testing process) from requirement analysis to test closure.

- Requirement Analysis
- Test Planning
- Test Case Design
- Test Environment Setup – Process of preparing software, application, tools required for testing.
- Test Execution
- Defect Reporting – to the development team
- Test closure

Functional Testing

→ It checks whether the application features/functions are working correctly according to the business requirements.

ex → login, search, payment

Non-Functional Testing

→ It verifies how well the system performs

ex → performance, load, security, Usability

Verification ⇒ Checking documents without executing the application

ex ⇒ Reviewing requirements, Test cases,

⇒ Are we building the product Right?

Validation ⇒ Checking the actual application.

⇒ Are we building the Right product?

Test Case ⇒ A set of steps used to verify a functionality of an application.

Test Scenario ⇒ A idea of what to test.

Defect ⇒ It is a mismatch b/w expected and actual results.

★ Defect life cycle

- New
- Assigned
- Open
- Fixed
- Retest
- Verified
- Closed

⇒ It is the journey of a defect from identification to closure.

Severity = Impact of the defect on application functionality.

Priority = How quickly defect should be fixed.

Smoke Testing

- It checks whether the build is stable enough for testing.

(It is a basic check to ensure critical functionality are working).

Sanity Testing

- It verifies that a particular defect fix or functionality is working correctly.

(Checks a specific functionality after changes).

Retesting - Testing the exact defect after it is fixed.

Regression Testing

- Entire related application is tested after new changes (new + old).

Unit Testing ⇒ It is done by dev, Tests individual methods/classes.

Integration Testing ⇒ checks interaction between modules.

System Testing ⇒ Tests complete applicati

UAT(User Acceptance Testing) ⇒ Client veri requiremen

Black Box Testing ⇒ Testing th Functionality witho seeing the code (without knov of internal code)

White Box Testing ⇒ It is done developers, they know and verifies the

Grey Box Testing ⇒ Combination Partial knowledge

Positive Testing ⇒ Expected results should occur.

Negative Testing ⇒ System should handle errors properly.

Test Plan ⇒ It is a document describing how testing will be performed.

Entry Criteria ⇒ Conditions that must be completed before testing starts.

Exit Criteria ⇒ Conditions that must be met before testing ends.

Agile ⇒ It is a software development methodology where work is divided into small parts called sprints, and software is developed and delivered in small increments.

(multiple feedbacks are provided)

Product Owner ⇒ responsible for gathering and prioritizing business requirements

Scrum Master ⇒ It helps the team follow scrum principles.

Scrum ⇒ Agile framework.

Product Backlog ⇒ A list of all requirements

Sprint ⇒ A fixed time period in which work is completed.

User Story ⇒ A requirement written from the prespective of user.
(short description of a requirement)

Agile Ceremonies ⇒

- Sprint Planning — before the sprint starts
- Daily Stand-up (short daily meetings)
- Sprint Review — end of the sprint
- Sprint Retrospective — Discussion for Continous Improvement

Test Design → process of creating detailed test cases from test conditions.

Acceptance Criteria → are the conditions that a feature must satisfy before it is completed or accepted by the product owner or client.

→ Meeting at the end of the sprint to demonstrate the completed work.

→ meeting

Selenium Automation

- It is an open-source automation tool used to automate web applications.

Selenium components

- Selenium IDE ⇒ Record and Playback tool
- Selenium Webdriver ⇒ Automates browser using code supports Java, Python, C#
- Selenium Grid ⇒ Executes tests on multiple browsers and machines simultaneously.

- Webdriver ⇒ It's an interface that allows selenium to interact with web browsers.

- ex⇒ Webdriver driver = new ChromeDriver();
  (Launch Browser)

  Webdriver = Interface
  ChromeDriver = Implementation class

- Open Website ⇒ driver.get("url");
- Maximize Window ⇒ driver.manage().window().maximize();

Selenium Concepts ⇒ Selenium involves WebDriver, Locators, WebElement, XPath, Waits, Alerts, Frames & Actions.

- close Browser ⇒ driver.close();
  ⇒ It close current window only.
- Quit Browser ⇒ driver.quit();
  ⇒ It close all browser windows.

★ Locators ⇒ They are used to identify web elements.

- ID ⇒ driver.findElement(By.id("username"));
- Name ⇒ driver.findElement(By.name("username"));
- ClassName ⇒ driver.findElement(By.className("input"));
- TagName ⇒ driver.findElement(By.tagName("input"));
- Xpath ⇒ driver.findElement(By.xpath("//input[@id='username']"));
- CSS Selectors ⇒ driver.findElement(By.cssSelector("#username"));

★ XPath ⇒ Used to locate elements when other locators are not available

- Absolute Xpath ⇒ /html/body/div/input
  ⇒ It is not preferred.
  ⇒ ~~Slash~~ (Single slash /)
- Relative XPath ⇒ //input[@id='username']
  ⇒ double slash (//)

XPath Functions

- text() ⇒ //button[text()='Login']
- contains() ⇒ //button[contains(text(),'Log')]
- ~~starts~~-with() ⇒ //input[starts-with(@id,'user')]
- normalize-space() ⇒ //button[normalize-space()='Login']
  • Removes extra spaces before comparing text.

⚠ WebElement ⇒ It is an interface in selenium that represents a webElement on a webpage such as text, boxes, button, links.

WebElement Commands

- Send Text ⇒ driver.findElement(By.id("username")).sendKeys("Lakshay");
- Click ⇒ driver.findElement(By.id("login")).click();
- Get Text ⇒ String Text = driver.findElement(By.id("msg")).getText();
- Clear Text ⇒ driver.findElement(By.id("username")).clear();

---

findElement() ⇒ It is a method to locate the first webElement that matches the specified locator.
- Returns a single webElement

ex ⇒ WebElement element = driver.findElement(By.id("username"));

findElements() ⇒ It is used to locate all WebElements matching a specified locator.
- Returns a list of elements

ex ⇒ List<WebElement> elements = driver.findElement(By.tagName("a"));

Waits

- Waits are used to pause execution until a certain condition is met.

- **Implicit Wait** → It applied globally.
  - → It waits for a specified time for all elements throughout the script.

  Ex → driver.manage().timeouts().
  implicitlyWait(Duration.ofSeconds(10));

- **Explicit Wait** → Applied to a specific element.
  - → It waits until a specific condition is met.

- **Fluent Wait** → A type of explicit wait with polling frequency and exception Handling.
  it gives more control; set polling frequency; ignore specific exception.

Dropdown Handling

Select dropdown = new Select(driver.findElement(By.id("country")));
dropdown.selectByVisibleText("Text");

Alerts

Accept alert → driver.switchTo().alert().accept();

Dismiss alert → driver.switchTo().alert().dismiss();

Iframe

Iframe (inline frame) is an HTML element used to embed another document within the current page.

Selenium cannot interact with elements inside an iframe unless it switches into the frame context.

Switch inside frame → driver.switchTo().frame("frameName");

(Frames are HTML Elements that contain another webpage inside a webpage)

How to switch from a frame →
- using switch index
- switch by name or ID
- switch by webElement
- Return from frame using .defaultContent()

Window Handling ⇒ It is the process of switching control b/w multiple browser windows or tabs opened during test execution.

- getWindowHandle() ⇒ returns the current window ID.
- getWindowHandles() ⇒ returns IDs of all opened windows.
- switchTo().window() ⇒ Switches control to another window.

Actions ⇒ used for advanced mouse and keyboard actions.

- Mouse Hover ⇒ Actions action = new Actions (driver);
  action.moveToElement(element).perform();

Double Click ⇒ actions.doubleClick(element).perform();

Right Click ⇒ actions.contextClick(element).perform();

JavaScript Executor

⇒ Used when normal Selenium methods don't work. It is used to run JavaScript code directly in the browser from your automation script.

- ex:-

```
JavaScriptExecutor js = (JavaScriptExecutor) driver;
js.executeScript("Your javascript code", args);
```

TestNG

- TestNG (Test Next Generation) is a testing framework used with Selenium to organise, execute, and generate reports for test cases.

Why use TestNG?

- Without TestNG → every test needs a main() method or object calling inside a main() method.
- With TestNG → It provides annotations, reporting, grouping, priorities, dependency and test execution through testng.xml
  No main() method is required.

ex →
```
@Test  → annotation
public void loginTest(){
}
```

TestNG Annotations

It controls the execution flow of test cases.

- @BeforeSuite → Runs once before the entire test suite.
- @BeforeTest → Runs before test execution starts.
- @BeforeClass → Runs once before all test methods in a class.
- @BeforeMethod → Runs before every @Test method

- @test ⇒ Actual test case.
- @AfterMethod ⇒ Runs after every test methods.
- @AfterClass ⇒ Runs once after all test methods.
- @After Test ⇒ Runs after all test execution.
- @After Suite ⇒ Runs once after the entire suite.

- Testng.xml ⇒ Used to run one or more test classes.

It is used to control and execute test classes in TestNG.

Priority ⇒ Controls execution Order
priority = 1, 2, 3, = —

Execution = login, search, logout.

Depends On Methods ⇒ One test depends on another.

ex ⇒

```
@test
public void login() {
}
@test (dependsOnMethods = "login")
public void booking() {
}
```

Page No.
Date:

Groups :- Used to execute specific test cases according to their group.

ex :- @Test(groups = "smoke")
@Test(groups = "regression")

Assertions :- Used to validate expected and actual results.

ex :- Assert.assertEquals("Success", "Success");
Passes

("Success", "Failed");
Fails

Page No.
Date

Wrapper Class ⇒ It converts a primitive data type into an object.

Collections like ArrayList store objects, not primitive data types.

ex ⇒
```
int a = 5;
Integer b = a;
sout(b);
```

- == vs .equals() ⇒ == operator

checks whether two references point to the same objects

ex ⇒
```
String s1 = new String("Java");
String s2 = new String("Java");

sout(s1 == s2); false
```
⇒ because both objects are different in memory.

.equals ⇒ checks whether the actual values are the same

ex ⇒ `sout(s1.equals(s2)); //true`

because both contains Java.

Page No.
Date:

Iterator is an Interface.

- Iterator ⇒ It is used to read elements from a collection one by one.

ex ⇒

```
List<String> list = new ArrayList<>();
list.add("Lakshay");
list.add("Samar");

Iterator<String> it = list.iterator();
// checks whether the next element exists
while(it.hasNext()) {   // returns true or false
    sout(it.next());
    // Returns the next element
}
```

- Apache POI ⇒ It is a Java library used to read and write Excel files in Selenium projects.

With the help of Apache POI, we read the excel data values and passes them to selenium.

⇒ Apache POI is a Java API used to read & write in Excel files. In Selenium, Data-Driven framework to fetch test data from Excel sheets.

- Workbook ⇒ Entire Excel file

ex ⇒ Workbook wb = WorkbookFactory.create(file);

- Sheet ⇒ A sheet inside Excel

ex ⇒ Sheet sheet = wb.getSheet("TestCases");

- Row ⇒ Row row = sheet.getRow(1);

- ~~Column~~ Cell ⇒ Cell cell = row.getCell(0);

POM (Page Object Model)

⇒ POM is a Selenium design pattern where web elements and methods are maintained in seperate page classes to improve maintainability and reusability.

Without POM ⇒ Everything in one file.

With POM ⇒ Pages
- LoginPage
- SearchPage
- BookingPage

Tests
- LoginTest
- BookingTest

Utility
- ExcelUtils
- WaitUtils

- pom.xml → is a maven configuration file used to manage project dependencies, plugins and build information.

→ Use case → Instead of manually downloading jar files, Maven downloades them automatically using pom.xml.

(automatically downloads libraries inside the Maven Project)

- Maven → is a build and dependency management tool used in java project.

It helps manage :-

Project libraries (dependencies)

→ It automatically downloads required dependencies using pom.xml, manage project dependencies inside pom.xml, Organise project structure that called pom.structure.

(which is)

- When I don't know the topic or question?

⇒ I'm not sure about this topic right now, but i will work on it and improve my understanding during the upcoming phase of my training.

⇒ I'm not fully confident about this topic right now, but i will work on it.

Take Screenshot

```
TakesScreenshot ts = (TakesScreenshot) driver;
// Convert WebDriver into TakesScreenshot

File src = ts.getScreenshotAs(OutputType.FILE);
// capture Screenshot

File dest = new File("C:\\Screenshots\\google.png");
// Specify the location

FileUtils.copyFile(src, dest);
// Copy screenshot to the desired location
```